

Syntax as the growth of transparent semantic representations

An executable research workbench for Dynamic Syntax

[Classical semantics](#) / [Constructive semantics](#) / [TTR](#)

8 October 2026

ds-workbench.vercel.app

Updated on 8 October against public application/source snapshot 1aff525. The community repository is now github.com/StergiosCha/DS_workbench. This edition adds the re-designed workspace, explicit model controls, computational-rule playback and the searchable research library, alongside the earlier grammar extensions. The frozen coverage measurements retain their original dates. Present a research and teaching system with expanding fragments, not an arbitrary-text parser. A tagged workbench package release remains pending.

Built with AI assistance

Development tools and model access

Development

Astra with Codex

Fable 5.1 with Claude Code

Models in the application

OpenRouter provides access to all models used in the app.

Development used Astra with Codex and Fable 5.1 with Claude Code. OpenRouter provides access to the models used inside the application.

The research question

Can we broaden coverage while keeping derivations inspectable?

For a linguist

- Which word licensed each operation?
- Which requirements are still open?
- Where did this meaning come from?

For an engineering system

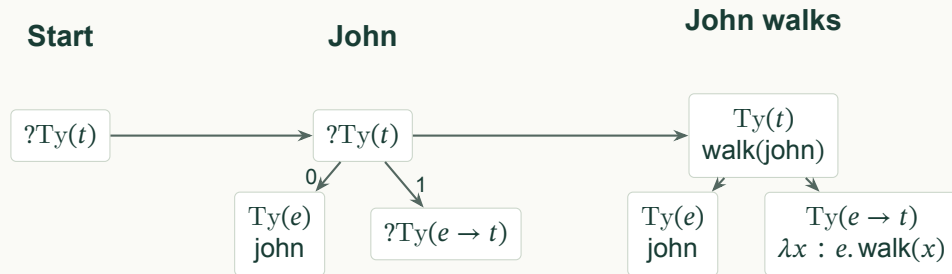
- Use dictionaries and corpora.
- Let models propose useful analyses.
- Retain failures and alternatives.

The DS engine must build the derivation. A model suggestion is an assumption to examine.

The motivation is the tension between a transparent formal fragment and ordinary open text. Three questions must stay separate: lexical coverage, formal completion, and linguistic adequacy. The interface is designed to expose rather than collapse them.

One word changes the state

A tree, a pointer, and requirements

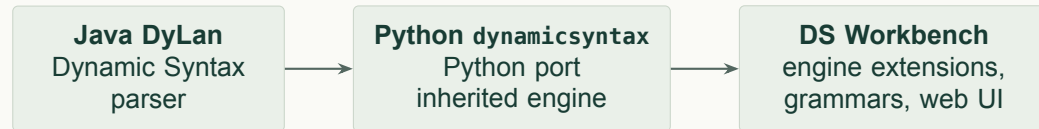


Schematic Classical states after local completion; intermediate pointer movements are omitted.

Question marks represent outstanding requirements, not probabilities. Lexical and computational rules build the daughters and compose by typed application. This drawing compresses intermediate steps; the application can show the individual make, go and put instructions. The context DAG stores alternatives and is different from this one semantic tree.

What the project is built on

Software foundation



- dynamicsyntax: public Python API, grammar discovery, results.
- dylan: most parser, tree, action and formula implementation.
- A DS parser with substantial TTR machinery is the foundation.

The two Python namespaces are part of one distribution, not separate servers. Installing the published package alone does not reproduce this modified source checkout, its resources or the deployed application.

What the workbench adds

	Inherited foundation	Extensions in this workbench
Parsing	Incrementality, action language, alternatives, context DAG	Recovery, limits, diagnostics, replay and state isolation
Semantics	TTR records and composition	Native Classical/Constructive terms; TTR identity corrections
Grammar	Bundled resources and induction infrastructure	English/Greek fragments; typed templates; supplemental relatives
Application	Existing interfaces and visualisation work	Focused browser views, live rule replay, research library and community source

The contribution includes changes to the engine and grammars, as well as the interface.

Avoid implying there was no previous GUI or visualisation. The current work builds on existing interfaces too. The focus here is the present integrated workbench, not a complete history of all upstream contributions.

Shared DS machinery, three semantic implementations

	Classical	Constructive	TTR
Representations	Predicates, λ , ε/τ choice terms	Typed witnesses, Σ/Π , projections	Record types, fields and dependencies
Composition	Typed application and reduction	Application, subtyping and witness closure	Record functions, paths and record merge
Current assistance	Native English / Standard Greek	Native English / Standard Greek	No construction or vocabulary compiler in this interface

Selecting a backend selects its grammar and semantic operations. It does not reformat a completed TTR parse.

The shared parser architecture does not mean identical trees, coverage or meanings. Classical common-noun restrictors and Constructive common-noun types have different internal structure. Coq export belongs to the supported Constructive path, not all three systems. The app’s TTR backend supplies record meanings to the Python DS executor. This is distinct from Cooper and Larsson’s TTR-DS recasting of DS states and actions inside TTR; see the dedicated appendix slide and library card CL25.

The same sentence, different semantic objects

A man knows you.

Classical

$$\text{know}(\varepsilon x : e.\text{man}(x), \text{you})$$

Constructive: witness supplied to the predicate

$$\Sigma p : (\Sigma x : \text{man}.\top). \text{know}(\pi_1(p), \text{you})$$

Constructive: simplified interpretation

$$\Sigma x : \text{man}.\text{know}(x, \text{you})$$

An interpretation type is not a constructed proof that the assertion is true.

These are the native outputs documented in the technical report; bound-variable spelling is simplified. Explain the projection: the indefinite introduces a witness package and its first component supplies the individual argument. Do not identify a Coq type-correctness check with establishing the sentence's truth.

TTR keeps participant and event dependencies explicit

x_0	$: e$
$p_0 = \text{man}(x_0)$	$: t$
$x_1 = \text{you}$	$: e$
$e_0 = \text{know}$	$: es$
$p_2 = \text{pres}(e_0)$	$: t$
$p_5 = \text{subj}(e_0, x_0)$	$: t$
$p_4 = \text{obj}(e_0, x_1)$	$: t$
$\text{head} = e_0$	$: es$

- Subject and object retain their own dependencies.
- Freshening must preserve constants and field identity.
- Native grammar extensions do not automatically extend TTR.

Readable presentation of the recorded TTR analysis; field equality here denotes a manifest field.

The earlier TTR bug allowed variable collisions to corrupt argument identity even when a tree completed. Branch-local freshening, constant preservation and semantic regression tests were needed. Different variable names do not assert that the referents are unequal: they preserve dependency structure. This motivates checking meaning, not just a green badge.

Coverage is a set of executable fragments

Native English

- Names, determiners and nominal constructions.
- Verb frames, copulas, PP arguments and modifiers.
- Relatives and finite clause relations.
- Limited questions, reflexives and dialogue repairs.

Native Standard Greek

- Case-sensitive names and nominal constructions.
- Clitics, clusters, verb frames and selected word orders.
- PP goals, comparisons and finite clause relations.
- Other varieties have smaller reviewed fragments.

Coverage is defined by supported constructions, not by the language label.

Do not claim that every determiner, relative, question or word order is supported. Number, agreement, tense, aspect and discourse remain partial. The dialect and historical displays have different evidential and executable scope.

A literature map now guides implementation

18 categories / 91 tracked topics / source-linked obligations

Families in the map	Examples of theoretical coverage
Growth and predication	Lexical programs, nominals, scope, valency and copulas
Dependencies	Flexible order, relatives, resumption, extraction and modification
Reference and dialogue	Coordination, binding, clitics, ellipsis, repair and grounding
Clauses and variation	Auxiliaries, tense/voice, clause linkage, agreement and historical change
Computation	Learning, search, generation and semantic architectures

An analysis in the literature is a starting point for a program and semantic tests. It is not automatically implemented here.

The initial citation-led coverage review on 6–7 October had 14 bibliography entries: 13 inspected artifacts and one metadata-only follow-up. The later selected-work library has 53 works and its own inspection records; these are different inventories, not competing counts. Long books and theses received targeted section review. The five rows group the eighteen categories for presentation. The map is not exhaustive. Distinguish worked analyses, reported implementations, proposals, secondary pointers and metadata from this code’s independent implementation status.

A searchable library connects papers to implementation

53 selected works / 30 source versions / 8 analysis cards

Find and inspect

- Open the app's **DS Library** tab.
- Search text; filter topic and semantic framework.
- See review scope and publication links.

Trace an analysis

1. Work and source version.
2. Bounded claim and exact passages.
3. DS mechanisms and semantic obligations.
4. Backend-specific code and tests.

Most readings are partial. A bibliographic record never establishes implemented coverage.

The app has a read-only, generated catalogue. Filtering it sends no model request and preserves the parse. The offline library in docs/research/library has the fuller version/evidence view, eight analysis cards and BibTeX/CSL exports. The 30 source versions are fingerprinted; this does not mean 30 full readings. Work, source version, bounded analysis and implementation are separate entities. A source PDF is not redistributed by the in-app catalogue. Zotero import files exist; shared-group editing and reviewed synchronization remain future work. Include semantic framework and TTR-integration tags so DS–TTR content semantics is not confused with the separate TTR-DS recasting.

PP arguments and modifiers do different jobs

Sentence	Implemented treatment
John relies on Mary.	Selected verb argument: an on-complement is required.
John gives a book to Mary.	Recipient argument in the dative-to frame.
John walks in a park.	Modifier grows through LINK; its own NP is parsed.
A man is in a park.	Copular locative predicate.

in_location(place, walk(john))

Schematic modifier meaning. The fragment does not supply a spatial or event-inference model.

Address the common intuition that modifiers should be LINK structures: these PP modifiers do use LINK. Selected arguments are different. A location PP cannot silently fill a missing recipient or selected preposition requirement. These are implementation choices within this fragment, not a claim that one analysis is the only possible DS treatment.

The same LINK mechanism can compose different meanings

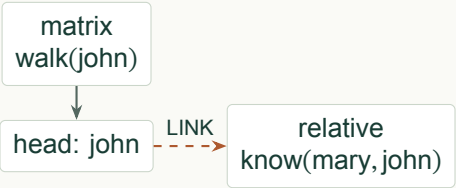
Construction	Example	Semantic obligation
Restrictive relative	A man who Mary knows walks.	Refine the nominal restrictor before quantification.
Nonrestrictive relative	John, who Mary knows, walks.	Preserve John; add a supplemental assertion.
PP modifier	John walks in a park.	Attach the locative content at the intended scope.

Specify what is copied, where requirements close, and how the linked content is evaluated.

The LINK relation is a structural resource, not a universal instruction to conjoin. The current PP treatment is the workbench’s declared implementation choice. The nonrestrictive follows the source’s initial conjunctive account, which is not a complete theory of projection or conventional implicature. The Classical and Constructive compilers must each satisfy the intended identity and scope contract; TTR requires its own audit.

From the source analysis to an executable relative

John, who Mary knows, walks. No LLM needed.



Schematic: functor branches and intermediate states omitted.

- 1. Comma opens a LINKed proposition.
- 2. *Who* copies the head to an unfixed node.
- 3. MERGE fixes the copy as the object of *knows*.
- 4. Close the relative; return and compose.

$$\text{walk}(\text{john}) \wedge \text{know}(\text{mary}, \text{john})$$

The matrix head at 00 links into 00L. *Who* copies its referent to L*, and the existing merge-relative rule fixes that copy at the relative subject L0 or object L10; here it is the object. Closing comma checks the relative and returns to the head. Completion-time evaluation conjoins the propositions after matrix composition. The head remains the same constant. The graphic is a compressed dependency schematic, not a screenshot or a full binary derivation. Both native backends produce the displayed formula. Classical follows the cited conjunction analysis; the Constructive typed conjunction/witness treatment is our implementation adaptation, not a claim that the source provided that exact code.

Test the relative’s meaning and its boundaries

Implemented in both native English backends

- Named subject or direct-object hosts.
- Subject or direct-object gaps.
- Local negation, adverbs and indefinite witnesses.

Still outside this claim

- Quantified or complex definite hosts.
- Nonlocal gaps and embedded projection.
- Greek relatives and TTR parity.

56 focused tests; 8 public browser checks: six complete derivations and two expected incomplete cases.

7 October: authored construction checks, not an estimate of unseen-text accuracy.

The tests check exact formulas, head identity, roles, local witness scope, LINK/unfixed/MERGE operations, alternatives and Coq exports when the compiler is available. The two negative public checks use John, who Mary walks, arrives: there is no compatible object gap. They are implementation contrasts, not independently elicited judgments. Who/which animacy is not yet enforced. Polar questions, causal/temporal/conditional combinations and attitudes require a projection account before support can be claimed. The public record is data/coverage/nonrestrictive-public-2026-10-07.json, with no model calls.

Greek clitics: order, case and tree growth

της τον έδωσε. “S/he gave him to her.”

- Genitive recipient and accusative theme.
- Unfixed structure, MERGE and substitution.
- Ditransitive frame preserves three participants.

Readable role structure

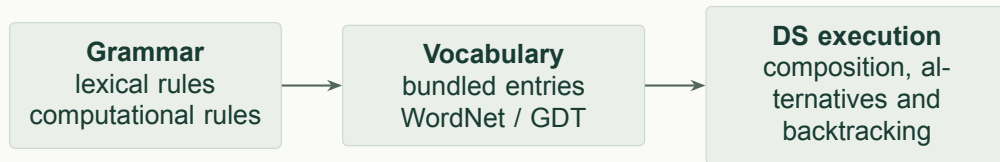
give(pro, him, her)
subject theme recipient

Predicate renamed for readability; pronoun reference is assumed.

PCC control: *του με έδωσε* fails in the modeled SMG clitic reading, in both native backends.

The formula renames the generated predicate while preserving roles. Genitive needs a recipient slot; a transitive lexical entry previously left it unresolved. The PCC card links Chatzikyriakidis’s thesis entry (6.49) and the 2011 article: genitive and first/second-person accusative programs compete for a locally unfixed address, while third-person accusatives have a fixed strategy. An unfixed node in a prefix is legitimate; the full PCC cluster must not receive a complete result. The user-supplied control is a regression example, not an independently sourced sentence judgment or a restriction generalized to all dialects and strong pronouns. Corpus frames remain hypotheses. Historical panels are not executable historical grammars.

What already works without an LLM

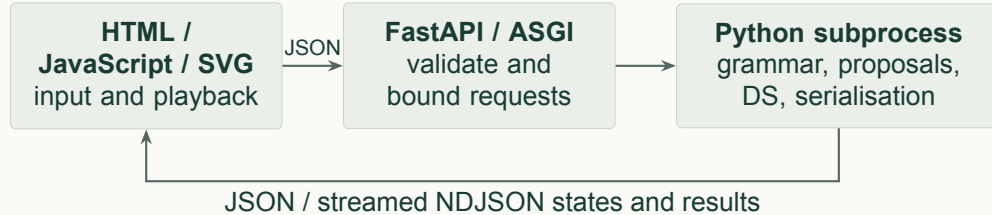


- Tree growth, meaning composition, diagnostics and playback.
- Sentence, paragraph and supported dialogue flows.
- Corpus/dictionary investigation and exports.

A model key is optional. Dictionaries still supply candidates, not guaranteed contextual meanings.

In dictionary or corpus mode there is no lexical model request. Corpus investigation may involve a live source service, but that is not an LLM call. A fully supported sentence can complete without a model in assisted mode too, except that ambiguous connective selection may deliberately request a contextual preference.

From the browser to an isolated DS worker



- Process isolation protects mutable parser and metavariable state.
- The browser renders recorded states; it does not run the parser.

The local standard-library HTTP server and hosted ASGI application use the same parser API. Request, response, worker and concurrency limits are part of deployment. The model key stays in the tab until refresh/disconnect, travels through the DS server for requests, and is not saved by the app. Avoid claiming that process isolation itself proves the implementation correct.

The model and the engine have distinct responsibilities

	Selected chat model	DS engine and compiler
Vocabulary	Propose lemma, sense, morphology and argument frame	Validate fields and instantiate an allowed lexical template
Constructions	Select a declared clause relation; propose a connective use	Instantiate a reviewed construction and compose its contents
Failure	Revise a hypothesis using actual DS feedback	Retry within limits; retain incomplete results

Model output contains structured hypotheses. It does not contain executable code or a finished replacement tree.

This is the central architectural change. Previously the model could only propose content words; conjunctions were excluded. The new interface supports construction hypotheses while keeping the underlying grammar operations reviewed. A model may still propose the wrong interpretation even if every schema and type check passes.

Two switches choose what the next parse may use

Use analysis LLM / Use Jev

LLM	Jev	Enabled route
OFF	OFF	DS with the selected original, bundled, dictionary or corpus vocabulary.
ON	OFF	Bounded lexical and construction assistance when needed.
OFF	ON	Jev word-meaning preferences; no generative fallback.
ON	ON	Jev preferences plus lexical-model fallback; no construction assistance in this combined mode.

Connect / models sets up access. The switches authorize use.
This result reports the requests actually made.

Both switches default off. Connecting or choosing a model does not enable either, and switching does not start a parse. An enabled LLM can make zero calls when the available grammar is sufficient. The result counters describe the displayed parse even if the settings change afterward. Jev sense assistance is native-English only; neither model assists TTR here. English dialogue supports lexical proposals, not open-text construction assistance. Jev’s advanced search-order control is distinct and still calibration-gated; Turn off all AI also resets it. No key is needed for grammar-only work. Model calls use the visitor’s OpenRouter account, and input goes to the selected provider.

A construction family replaces one-off conjunction fixes

Surface word → declared relation → shared DS program

Family	English examples	Greek examples
Cause	because; causal since	επειδή, διότι; causal αφού
Time	when, while, before, after, until	όταν, όποτε, ωστόσο
Condition	if, unless	αν, εάν
Concession / contrast	although, though, whereas	παρότι, μολονότι; readings of ενώ

Postposed clauses grow through LINK. Fronted clauses require an explicit comma and a main clause. Ambiguous words retain alternatives.

There are twelve relation IDs, not one generic conjunction that erases distinctions. The model can choose among declared readings or add a previously absent single-word lexicalization. The table lists supported finite-clause uses only: nominal after, interrogative when, multiword markers and nonfinite complements are not licensed by these templates.

A recorded example of model assistance

John walks provided Mary walks.

Before assistance

The available lexical programs do not license this connective use.

Model proposal

```
surface: provided
relation: condition
```

Actual DS result

```
condition(walk(john),
           walk(mary))
```

The UI exposes the compiled template and the relation on the accepted DS path.

Recorded 4 October checks: *provided* and Greek *καθότι* completed in both native backends.

These were real OpenRouter calls followed by DS parsing, recorded on 4 October, not new calls made for this slide update. For *καθότι*, the missing-word request returned a construction directly instead of exhausting the retry budget on noun/verb guesses. The public browser check also verified provided, export and key privacy. These demonstrations do not estimate general model accuracy.

Jev has a different, visible role

John lends a book to Mary.

DS without Jev preferences

“bestow a quality on”

A complete but contextually poor lexical-sense choice.

DS with Jev preferences

“give temporarily”

DS replay accepts a preferred sense after more context arrives.

- Two actual runs use identical lexical programs.
- Jev ranks senses from the observed prefix; alternatives survive.
- Its pinned model is separate from the selected chat model.

Recorded demonstration, not an accuracy or speed benchmark. The independent search-order control remains gated by calibration.

Show the distinction between semantic adequacy and formal completion: both parses completed. Jev’s early response may be uncertain and outcomes vary across calls. The controlled comparison freezes candidates and semantic profiles; simply switching dictionary modes would not be a fair comparison because their candidate order differs. Comparison currently supports native English sentences only.

Corpora and dictionaries supply evidence

Lexical resources

- English: reviewed additions; optional WordNet index.
- Greek: GDT training export.
- 2,021 Greek forms; 2,330 analyses.

Optional live Greek evidence

- Svarna examples and corpus context.
- Triantafyllidis headwords and excerpts.
- Observations and citations accompany lexical proposals.

An attested occurrence is evidence for an analysis. It is not an exhaustive valency specification or automatic grammar training.

Live parsing integration is opt-in and bounded, for native Standard Greek sentence/paragraph assistance. Retrieved IDs are validated, but citation relevance is not automatically proven. Held-out GDT material is separate from the training builder. Explain the independent source licenses if the audience asks about redistribution.

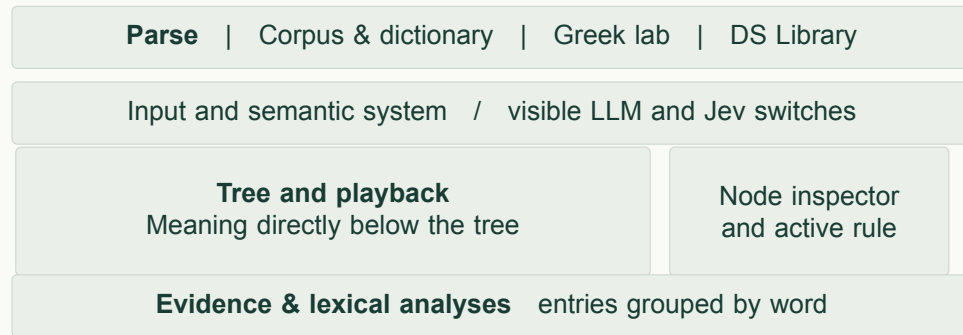
Sentences, paragraphs and dialogue

Input	State handling	Important boundary
Sentence	Incremental parse and alternative complete analyses	Completion is relative to the selected grammar
Paragraph	Completed sentences retain DS context	Failures count and become explicit context gaps
Dialogue	A new speaker can continue the same tree; supported repairs preserve history	General ellipsis, grounding and discourse inference remain incomplete

A later successful sentence does not turn an incomplete paragraph into a complete interpretation.

The interface supports up to 400 paragraph tokens and 24 sentences, and up to 12 dialogue turns and 80 tokens. Basic name-based pronoun resolution is not a general discourse model. Open-text construction assistance applies to sentences and paragraphs; it is not the dialogue assistance pipeline.

A workspace organised around the growing tree



Settings open in a drawer. Tabs preserve the parse. Manual zoom persists; Auto-fit resumes following the tree.

This is a layout schematic, not an application screenshot. The new desktop view places the tree closer to input. Three examples are shown initially; All examples expands them. Parse options and coverage are a disclosure. On mobile, larger playback targets and a wrapping timeline keep controls accessible; meaning stays below the tree and Inspect this step opens the inspector. A node click pauses playback and opens its details. Evidence follows the tree, with candidate programs grouped by word and reported selections first. Inspect failure opens the evidence. Tabs alone make no parser/model request; asking DS to analyse a corpus or Greek-lab example returns to Parse. Fullscreen and side-by-side semantic comparison are not implemented by this redesign.

Computational rules appear while the tree grows

Rules is the default playback level

See the active program

- Lexical and computational rules in execution order.
- Introduction and anticipation; completion and elimination.
- Pointer movement and changed decorations.

Choose the level of detail

- **Words:** states at word boundaries.
- **Rules:** each lexical/computational action.
- **Operations:** make/go/put inside a rule, where recorded.

Backtracking restores a prior state. **Show final result** returns to the endpoint.

In the current grammar, intro-pred combines Introduction/Prediction by creating argument and predicate daughters with type requirements. Anticipation0/1 separately move the pointer to children with open requirements. These are DS grammar actions, not model predictions; names and schedules differ across grammars. Rule-level events are emitted as parsing advances for ordinary, assisted and paragraph paths; the browser replays recorded states. This is not a debugger over every rejected branch. Assisted retries restart at a labelled axiom; paragraph rows have independent traces. Ordinary sentence/dialogue traces can show atomic instructions, while assisted/paragraph traces use compact rule-level snapshots. Bounded hosted traces may make an explicitly labelled jump to the final state. A partial playback frame does not contradict a complete final result. JSON and supported Constructive Coq export remain available; the public server does not execute Coq.

Evidence for coverage: keep the samples separate

Check	English	Greek
Unseen-source baseline (1 Oct.)	0/6 paragraphs	0/6 passages
Hand-written assisted development set	6/9 sentences	5/9 sentences
New clause paragraphs	3/3 sentences	3/3 sentences
Live new-connective examples	provided completed	καθότι completed

- Results above were checked in both native backends.
- The frozen baseline predates the latest extensions.
- These small samples are not a representative accuracy estimate.

The old unseen-source baseline is intentionally retained rather than silently replaced by successful demos. Greek passages group three consecutive test sentences; original paragraph boundaries were unavailable. The new three-sentence examples use dictionaries/corpus candidates without an LLM. Tests of known examples establish regression behavior, not open-text coverage.

What “complete” does—and does not—establish

It establishes

- A path through the implemented DS programs.
- Satisfied tree requirements and available type checks.
- A composed meaning under displayed assumptions.

It does not establish

- Correct sense, reference or attachment.
- Independent grammaticality or truth.
- Full temporal, causal or counterfactual reasoning.

Clause relations are distinct, typed, opaque predicates. That is useful structure, but still a limited semantics.

Condition(main, subordinate) is not silently treated as material implication. A later modifier once erased a nested clause in an alternative reading; the regression check had to assert the actual formula, not only completion. Type systems help expose errors but do not prove this Python implementation or its grammar correct.

A resource other DS researchers can use and extend

Starting point	What is available	A useful contribution
Explore / teach	Browser, traces, examples and slides	An exercise or sourced counterexample
Reproduce	Local install, Python API, exported results	Pinned input, grammar and expected meaning
Extend	Independent grammar folders and a tested tutorial	A lexical entry or a construction family
Maintain	Builders, tests, guides and packaging checks	Review, releases and language-specific stewardship

The app and grammar tutorial work without a model subscription or the original developer’s machine.

Source, issues and pull requests are at github.com/StergiosCha/DS_workbench. The public snapshot preserves inherited history and adds the workbench implementation. The community guide separates practical use, linguistic evidence, grammar work and maintenance. The wheel includes static assets and selected runtime data; clean installation outside the checkout passed HTTP/streaming and tutorial checks. Large WordNet/corpus indexes remain optional. Source publication is complete; a tagged workbench package, contributor terms and broader maintainer responsibilities remain future work. Installing the upstream PyPI package by name does not give this snapshot.

Next: extend families, then measure generalisation

1. **Reference and relatives.** Source a Greek *που* fragment; audit TTR identity and roles; specify supplement projection.
2. **Predicate reuse and ellipsis.** *John knows Mary. Bill does too.* Test antecedents and strict/sloppy readings.
3. **Nominals and events.** Coordination, possessives and plurality; then auxiliaries/voice with explicit event semantics.
4. **Community and evaluation.** Review analysis cards, tag a release, freeze fresh English/Greek samples and publish all outcomes.

Each extension needs positive examples, negative contrasts, meaningful output checks and reproducible evidence.

These priorities follow the literature map. The first bounded English nonrestrictive increment is implemented; the Greek relative, broader ellipsis, nominal and event work listed here is future work. Ellipsis needs an accessible typed predicate history and freshening before a model can rank compatible antecedents. The full programme also includes questions, agreement, clause semantics and wider typology. Reevaluate on a fresh held-out sample; do not train on failed baseline examples and continue calling them unseen. Each backend has its own semantic obligations.

A short demonstration

Open DS Workbench / start in Parse

1. **Without AI** John, who Mary knows, walks.
Leave both switches OFF; follow Rules, LINK and MERGE.
2. **Construction assistance** John walks provided Mary walks.
Connect privately; LLM ON, Jev OFF. Inspect the relation and actual usage.
3. **Jev comparison** John lends a book to Mary.
LLM OFF, Jev ON; open *See Jev in action* and compare.
4. **Find the research** DS Library → search PCC or CL25.
Inspect review scope; return to Parse with the derivation preserved.

Allow four to six minutes. Use native English and Classical or Constructive. The relative uses bundled original entries; to force Original lexicon use Connect / models, Advanced, Vocabulary source, then close the drawer. Step in Rules, expand a formula and show the final result. John, who Mary walks, arrives is an incomplete control. For provided enable only the analysis LLM: combining both switches selects a different lexical route without construction assistance. For lends enable only Jev and open the comparison below the evidence area, or use See Jev in action. Do not expose an API key on the projector. No new live model evaluation was run for this slide update: if the provider is unavailable, use the recorded demonstrations and identify them as recorded. The library needs no model or key.

Appendix: DS–TTR and TTR-DS are different proposals

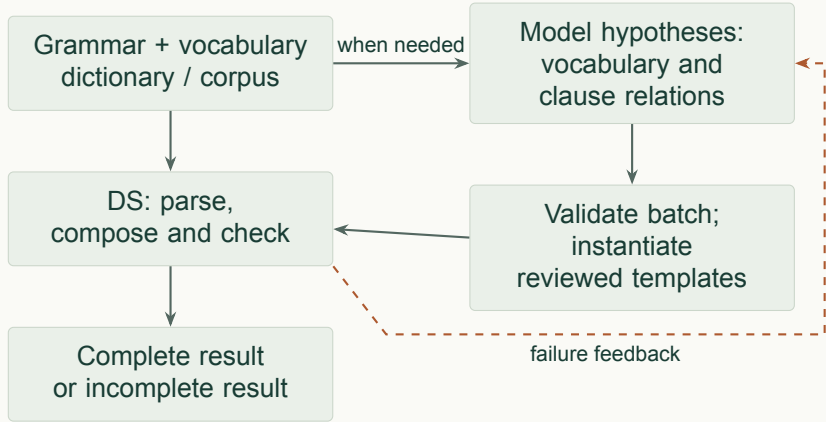
	DS–TTR content semantics	TTR-DS recasting
What TTR represents	Semantic records, fields and dependencies in DS processing	DS tree descriptions, parse states and action rules themselves
In this workbench	Inherited record-semantic machinery in a Python executor	Research card and prototype specification as future work
What follows	A separate TTR grammar must be tested on its own	The existing TTR dropdown is not an implementation of this recasting

A representation of DS in TTR does not require every semantic content to be a TTR record.

Cooper & Larsson (2025), §§1–2.2, 3, 4.8–5; DS Library record CL25 and linked analysis card.

The inspected proposal concentrates on classical e/t contents while representing the surrounding DS machinery in TTR. Further semantic integrations and a detailed implemented grammar are left as future work. Do not read the title as evidence of a new implemented backend or of parity with Classical and Constructive. Selected sections were inspected, not every formal rule. The library keeps this distinction in a separate TTR-integration facet. An experimental prototype would require its own state/action specification and semantic regression tests before integration into the app.

Appendix: bounded proposal–compile–parse loop



At most three model calls, shared across a paragraph; 48-second helper budget. Provider failures preserve available grammar alternatives.

Construction-only calls have a 16-second ceiling, vocabulary calls up to 22 seconds within remaining time. The worker boundary is 55 seconds and the hosted function 60 seconds. The model may inspect full context; this pipeline is not a strictly prefix-only cognitive model. Jev’s lexical prefix experiments have a different context contract.

Appendix: the model never supplies this program

Structured model output

```
{"constructions": [{
  "surface": "provided",
  "relation": "condition",
  "evidence": "Finite condition."
}]}
```

Historical operation names retain “causal”; the parameter selects the reviewed relation. Existing alternatives are retained.

Locally instantiated template

```
IF Ty(mltt:Prop)
THEN causal-post(condition)
ELSE abort
```

The atomic operation executes visible make/go/put steps, opens required content, and preserves local closure.

Validation restricts surface, relation and evidence fields; known connectives also have a reviewed reading inventory. Relation signatures cannot be invented. New lexicalizations remain hypotheses within the request, not persistent grammar training. Show the expanded program in the UI if asked.

Appendix: local content closure and Coq

A man walks if a dog walks.

$$\text{condition}(\Sigma x : \text{man.walk}(x), \quad \Sigma y : \text{dog.walk}(y))$$

Constructive declaration

```
Parameter condition : Type -> Type -> Prop.
```

- Each clause closes its own witnesses.
- Opaque relations accept content types in the supported exporter.
- Coq checks the exported representation; it does not prove the intended interpretation.

Bound variables are renamed for readability. The Type-level arguments allow dependent witness contents without pretending that every content has already been reduced to a simple extensional proposition. The current choice does not deliver a full proof-theoretic analysis of natural-language conditionals. Classical preserves its own choice-term contents.

Appendix: implementation details that matter

Mechanism	Why it matters
Mutable-state isolation	Parser workers and branch-local freshening protect unrelated analyses.
Atomic validation	An invalid batch cannot leave a half-installed grammar.
Scope and replay	Restore context and metavariable state before retries; check meanings after revisions.
Generated grammar sections	Update builders and emitted files together; check idempotence.
Resource and search settings	Resolve installed assets; preserve lexical alternatives with top_n=0 during coverage checks.

Code map: `workbench_api.py`, `assisted_parsing.py`, `construction_assistance.py`, `causal_effects.py`, `jev_comparison.py`.

These are not cosmetic details. Global metavariable pools can corrupt independent requests; stale ancestors can erase a relation after a modifier; mutable lexical inventories can invalidate a before/after comparison. The controlled Jev comparison hashes identical inventories and resets bindings between runs.

Appendix: verification of the current workspace

8 October 2026 / application source 1aff525

Check	Observed result
Local targeted Python tests	81 passed: API, ASGI, rule playback and readings.
Focused browser regres- sions	6 rule-playback cases; 19 PCC cases as expected.
Workspace browser checks	7 grouped checks passed on source, staged bundle and public app, including desktop/mobile and AI permissions.
GitHub CI at this commit	Python 3.11/3.12 and installed-package jobs passed; Python 3.13 cancelled at the time limit.

These are overlapping software checks. They do not update linguistic coverage or model-quality estimates.

Check scripts: `check_workspace_browser.py`, `check_rule_playback_browser.py` and `check_pcc_browser.py`. Workspace checks cover tab persistence, library filters, Greek examples, lexical groups, zoom, explicit model permissions, paragraph/dialogue and mobile bounds. Model connection tests use fixtures; no fresh billed model evaluation is claimed. CI run 37753914626 is not an all-green run: its 3.13 job reached the 45-minute job limit; the other three jobs passed. Earlier commit 02229a9 had a successful full CI run. Keep the historical 7 October audit in docs/community/handover-checks.md: 1,862 passed, three skips, four initial failures followed by focused corrections/environment checks. These counts are different runs and must not be added. Optional Coq/TeX checks need their dependencies; inherited TTR loading still needs a whole-grammar audit.

Appendix: foundations and further reading

- **Software.** Java DyLan, Dynamics of Language; Python `dynamicsyntax`.
- **Core DS.** Cann, Kempson & Marten (2005), *The Dynamics of Language*; Howes & Gibson (2021), *JoLLI* 30, 263–276.
- **Constructive semantics.** Chatzikyriakidis (2025), *Constructive Dynamic Syntax, Languages* 10(11):269.
- **Greek clitics.** Chatzikyriakidis (2010), thesis, chapters 3 and 6; Chatzikyriakidis (2012), *Lingua* 122(6), 642–672.
- **Project reports.** Complete technical walkthrough; theoretical-coverage catalogue; community contribution and maintenance guides.
- **Application and source.** [Public workbench](#); [community repository](#).

The slides describe the modified workbench snapshot. They do not claim complete implementations of the cited linguistic theories.

The repository’s technical report links the individual design documents, source code, data artifacts and software lineage. This is a selected reference list for the presentation, not a comprehensive bibliography of Dynamic Syntax or TTR.

Appendix: the complete 18-category map

C01	Incremental growth and lexical programs	C10	Auxiliaries, restructuring, tense/voice
C02	Nominals, reference, quantification, scope	C11	Causal, temporal, conditional clauses
C03	Predication, valency and copulas	C12	Ellipsis and fragments
C04	Flexible order, case and discontinuity	C13	Questions, split turns, repair, grounding
C05	Relatives, resumption, extraction, islands	C14	Information structure and peripheries
C06	Adjectives, adverbs, PPs and comparison	C15	Agreement, plurality and typological tests
C07	Coordination and shared material	C16	Historical development and variation
C08	Anaphora and binding	C17	Learning, search and generation
C09	Clitic placement, clusters and person	C18	Semantic architectures and interfaces

Full catalogue: docs/research/ds-theoretical-coverage.json and companion report.

The shortened slide headings correspond to the eighteen catalogue categories, including concessive linkage in C11 and aspect in C10. The 91 topics are tracked research units, not 91 complete grammar implementations. Evidence labels distinguish worked analyses, implemented studies, sketches, secondary pointers and metadata-only records. Implementation labels separately distinguish tested fragments, partial infrastructure, absent native features and unaudited inherited capabilities. The ledger contains 13 inspected artifacts with hashes and one metadata-only 2026 follow-up.

Appendix: extend an independent grammar

Copy the grammar; add a predicate and lexical entries; check meanings.

Semantic signature

dance : human $\rightarrow$ Prop

Lexicon entries

dances intransitive dance human
dance intransitive-base dance human

Public Python API

```
from pathlib import Path
from dynamicsyntax import parse
result = parse("Mary dances.",
               Path("/path/to/my-grammar"),
               strict=True, top_n=0, trace=True)
print(result.semantics)
```

Classical uses $e \rightarrow t$; both reuse the existing DS templates.

12 checks across both native backends; saved grammar hashes and expected meanings.

[examples/community/extend_grammar.py](#); [docs/community/extending.md](#).

This is a lexical extension using existing syntax, not a new construction family. The script copies packaged resources and refuses to overwrite an existing destination. It checks two names, negation, the nonrestrictive and two incomplete/extra-argument contrasts in each backend. Run with `–output` pointing to a new directory. `top_n=0` retains lexical alternatives; the inherited default of three can prune a necessary function-word/punctuation entry. Search caps still apply. Custom grammar folders load through Python; arbitrary uploads are not accepted by the public web interface.